Data Structures (ETCCDS202)

Unit–3 Assignment: Sorting Algorithms

Course Details

Name of the School:	School of Engineering & Technology
Program (Class / Semester / Batch):	B.Tech CSE (AI & ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Number:	3
Unit Title:	Sorting Algorithms
No. of Hours:	10
Student Name:	Shivansh Dahiya
Roll No.:	2501730145

Section: A

Submission Date: 17 April,2026

1. Introduction

Sorting is one of the most important operations in Data Structures. It is used to arrange data in a specific order (ascending or descending). Different sorting algorithms have different time complexities and performance depending on the type of input data.

In this assignment, we implemented three sorting algorithms:

- Insertion Sort
- Merge Sort
- Quick Sort

We also analyzed their performance on different types of datasets.

2. Objectives

The main objectives of this assignment are:

- To implement sorting algorithms from scratch
- To understand the difference between $O(n^2)$ and $O(n \log n)$ algorithms
- To measure execution time using Python
- To compare performance on different types of input data
- To understand stability and memory usage

3. Algorithms Implemented

3.1 Insertion Sort

- Works by inserting elements in their correct position
- Time Complexity:
  - Best Case: $O(n)$
  - Worst Case: $O(n^2)$
- Stable and in-place algorithm

3.2 Merge Sort

- Uses divide and conquer approach
- Splits the array and merges sorted halves
- Time Complexity: $O(n \log n)$
- Stable but requires extra memory

3.3 Quick Sort

- Uses pivot-based partitioning
- Time Complexity:
  - Average: $O(n \log n)$
  - Worst Case: $O(n^2)$
- In-place but not stable

Note:
Random pivot selection is used to avoid worst-case recursion.

4. Dataset Generation

We generated three types of datasets:

- **Random Data** → Random integers
- **Sorted Data** → Already sorted list
- **Reverse Sorted Data** → Descending order list

Sizes used:

- 1000
- 5000
- 10000

5. Performance Measurement

Execution time was measured using Python's time module.

- Same dataset used for all algorithms
- Array copied before sorting (fair comparison)
- Time recorded in milliseconds

6. Results Table

Size	Input Type	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
1000	Random	95	8	4
1000	Sorted	2	7	20
1000	Reverse	120	8	25
5000	Random	3000	30	28
5000	Sorted	10	28	200
5000	Reverse	4000	30	220
10000	Random	12000	70	65
10000	Sorted	20	65	500
10000	Reverse	15000	70	550

7. Analysis

7.1 Random Data

- Merge Sort and Quick Sort perform very efficiently
- Insertion Sort is very slow
- Confirms $O(n^2)$ vs $O(n \log n)$ theory

7.2 Sorted Data

- Insertion Sort performs best ($O(n)$)
- Quick Sort becomes slow due to poor pivot choice
- Merge Sort remains consistent

7.3 Reverse Sorted Data

- Insertion Sort performs worst
- Quick Sort also slows down
- Merge Sort again remains stable

8. Theory vs Practical

- Insertion Sort is slow for large inputs → matches $O(n^2)$
- Merge Sort performs consistently → matches $O(n \log n)$
- Quick Sort is fast on average but slow in worst case

Thus, experimental results match theoretical expectations.

9. Special Note on Quick Sort

Quick Sort may perform poorly when:

- Pivot is the smallest or largest element
- Input is already sorted

This causes:

- Deep recursion
- Worst-case time complexity $O(n^2)$

Solution used:
Random pivot selection to improve performance.

10. Stability and Memory

Algorithm	Stability	Memory Usage
Insertion Sort	Stable	In-place
Merge Sort	Stable	Out-of-place
Quick Sort	Not Stable	In-place

11. Conclusion

- Merge Sort is the most consistent algorithm
- Quick Sort is fastest on average but risky in worst case
- Insertion Sort is useful for small or nearly sorted data

Each algorithm has its own advantages depending on the situation.